

EJERCICIO PATRÓN TEMPLATE Y STRATEGY

EJERCICIO

Diseñar y desarrollar un programa en Java que permita construir distintos tipos de ordenadores utilizando:

- Patrón Template Method (obligatorio)
- Patrón Strategy (opcional, para variar el tipo de ordenador)

El ordenador debe constar de los siguientes componentes:

- Procesador
- Memoria RAM
- Almacenamiento
- Tarjeta gráfica
- Fuente de alimentación

El programa debe mostrar por consola cada paso del proceso de construcción y un mensaje final indicando que el ordenador ha sido ensamblado correctamente.

Parte 1 - Patrón Template

Se debe implementar una clase abstracta *ComputerBuilder* que defina el flujo general de construcción de un ordenador.

Requisitos:

- Método final *buildComputer()* que define el algoritmo completo:
 1. Preparar placa base
 2. Instalar procesador
 3. Instalar memoria RAM
 4. Instalar almacenamiento
 5. Instalar tarjeta gráfica
 6. Instalar fuente de alimentación
 7. Verificar ensamblado
- Los pasos específicos de instalación de componentes deben declararse como métodos abstractos:
 - `addProcessor()`
 - `addRAM()`
 - `addStorage()`

Parte 2 - Uso opcional de Strategy

Para permitir diferentes configuraciones de ordenador (por ejemplo Gaming, Office o Workstation), se puede aplicar el patrón *Strategy*.

En lugar de crear múltiples subclases de *ComputerBuilder*, se puede:

- Definir una interfaz *ComputerConfigurationStrategy*
- Delegar en ella la elección de componentes

Esto permite cambiar la configuración sin modificar el algoritmo de construcción.

Construcción del ordenador

En el método *main()*:

- Crear una configuración concreta (por ejemplo *GamingConfiguration*)
- Crear una instancia concreta de *ComputerBuilder*
- Ejecutar el método *buildComputer()*

Se pide:

- Diagrama UML de toda la solución.
- El código fuente de toda la aplicación. El programa debe **imprimir en consola cada paso realizado durante la construcción del ordenador**, así como un mensaje final indicando que el ordenador ha sido ensamblado correctamente.